■ Computer Vision — CAT II Study Notes

Feature Extraction • Image Segmentation • Pattern Analysis • Motion Analysis • ■ All Exam Topics

[UNIT 01] Feature Extraction — Edges, Lines, Corners, Descriptors

[■ EXAM HOT] Edge Detection — Canny, LoG, DoG

DEFINITION

An edge is a significant local change in image intensity. Edge detection finds boundaries by locating pixels where intensity changes sharply.

1. Gradient Basics

$Gx = \partial I/\partial x \; Gy = \partial I/\partial y \; |G| = \sqrt{(Gx^2 + Gy^2)} \; \theta = arctan(Gy/Gx)$

- **|G| (Magnitude)** — HOW STRONG the edge is; higher = stronger edge
- θ **(Orientation)** — which DIRECTION the edge goes (horizontal, vertical, diagonal)

Canny Edge Detector [Most Asked]

1. Gaussian Blur — smooth image to remove noise: G(x,y) = $(1/2\pi\sigma^2)\cdot e^{-(x^2+y^2)/2\sigma^2}$
2. Compute Gradient |G| and direction θ using Sobel operators
3. Non-Maximum Suppression (NMS) — keep only local maxima → thin edges
4. Double Thresholding — strong (>T_high), weak (T_low<p<T_high), none
5. Edge Tracking by Hysteresis — keep weak only if connected to strong pixels

```
Input→[Gaussian Blur]→[Sobel: Gx,Gy→|G|,θ]→[NMS: thin edges]→[Threshold]→[Hysteresis]→Edge Map
```

■ Pros	■ Cons
• Best noise robustness	• Slow (5 steps)
• Thin, precise edges	• Threshold tuning needed
• Two-threshold → fewer false edges	• Misses blurry edges

LoG (Laplacian of Gaussian)

$LoG(x,y) = -(1/\pi\sigma\blacksquare)[1-(x^2+y^2)/2\sigma^2]\cdot e^{-(x^2+y^2)/2\sigma^2}$

1. Smooth with Gaussian (noise removal)
2. Apply Laplacian (2nd derivative) → find zero-crossings
3. Zero-crossings = edge locations

DoG (Difference of Gaussians) — used in SIFT!

$DoG(x,y,\sigma) = G(x,y,k\sigma) - G(x,y,\sigma)$ *[fast approximation of LoG]*

- Approximates LoG — much faster to compute
- Subtracts two blurred images at different scales
- Used in SIFT keypoint detection across scale space

Method	Noise Handling	Speed	Localization	Best For
Canny	■■■ Excellent	Slow	■■■ Excellent	General edges, cameras
LoG	■■ Good	Medium	■■ Good	Blob detection, zero-crossing
DoG	■■ Good	■■■ Fast	■■ Good	Scale-space, SIFT keypoints
Sobel	■ Poor	■■■ Very Fast	■ Poor	Quick gradients only

■ Exam Tip (Q1 Mock)

Lane detection: |G| finds WHERE the edge is (high gradient=lane line), θ gives DIRECTION of lane. Canny recommended for autonomous vehicles due to noise robustness.

[Line Detector] Hough Transform — Line & Circle Detection

DEFINITION

Maps edge points from image space to parameter (Hough) space. Points lying on the same line produce peaks in Hough space.

$\rho = x\cdot cos(\theta) + y\cdot sin(\theta)$ *[Normal form of a line]*

1. Apply edge detector (Canny) to get edge map
2. For each edge pixel (x,y), vote for ALL possible (ρ,θ) pairs
3. Accumulate votes in 2D Hough accumulator array H[ρ][θ]
4. Find peaks in H — each peak = a line in image
5. Back-project peaks to image space to draw detected lines

```
Image Space Hough Space (ρ,θ) y■ * * * * ρ■ ■ (peak = line) ■ * * ■ ■ ■■■■■■■■■■■■ x ■■_____________ θ Each point → sine curve; Intersection = common line
```

Circle Hough Transform

$(x-a)^2 + (y-b)^2 = r^2 \rightarrow$ *Vote in 3D space (a, b, r)*

■ Pros	■ Cons
• Robust to noise & missing points	• High memory for 2D/3D accumulator
• Detects multiple lines simultaneously	• Slow for large images
• Works with partial/occluded lines	• Only detects predefined shapes

■ Exam Use

Robot warehouse shelf detection: Canny → Hough (straight lines) → Harris (corners). This 3-stage pipeline is frequently tested!

[■ EXAM HOT] Corner Detection — Harris & Hessian-Affine

DEFINITION

Corners are points where intensity changes significantly in ALL directions. More stable and distinctive than edges for matching.

Harris Corner Detector

$M = \Sigma\, w(x,y)\cdot[Ix^2 \; IxIy; \; IxIy \; Iy^2] \; R = det(M) - k\cdot trace(M)^2$

- **M** = Structure tensor (2×2 gradient matrix); λ■, λ■ = eigenvalues of M
- Both large → **Corner** (R > threshold) | One large → **Edge** | Both small → **Flat** (R≈0)
- k = 0.04 to 0.06 (empirical constant)

```
λ■ ■ Corner Edge ■ ■■■■■■■■■■■■■■■■■■■■■■■■ λ■ ■ Flat Edge ■
```

Hessian-Affine Detector

$H(x,y) = [Ixx \; Ixy; \; Ixy \; Iyy] \; det(H) = Ixx\cdot Iyy - Ixy^2$

- Uses 2nd-order derivatives (curvature information)
- Detects BLOB-like regions, not just corners
- Affine invariant — handles viewpoint changes
- More stable than Harris under scale/rotation

Feature	Harris	Hessian-Affine
Invariance	Rotation ■, Scale ■	Rotation ■, Scale ■, Affine ■
Speed	Fast	Slower
Detects	Corners	Blobs + corners
Use case	Indoor scenes	Wide-baseline matching

■ Exam Connection (Q2 Mock)

SIFT uses Hessian matrix in localization to eliminate unstable keypoints on edges (one eigenvalue >> other = edge response, not blob → REJECT).

[■ MOST IMPORTANT] SIFT — Scale Invariant Feature Transform

DEFINITION

Detects and describes local features that are invariant to scale, rotation, and partially to illumination and viewpoint changes.

4 Stages of SIFT

1. **Scale-Space Extrema Detection:** Build DoG pyramid at multiple octaves. Find local minima/maxima in 3D (x,y,scale) — keypoint candidates.
2. **Keypoint Localization & Filtering:** Taylor expansion to refine position. REJECT if: |DoG|<0.03 (low contrast), or Hessian ratio $(r+1)^2/r$>threshold → eliminates edge responses (r=10).
3. **Orientation Assignment:** Histogram of gradient orientations (36 bins, 360°) around keypoint. Dominant orientation = keypoint orientation → ROTATION INVARIANCE.
4. **Keypoint Descriptor:** 4×4 grid of cells, each with 8-bin histogram = 4×4×8 = **128-D vector**. Normalized to reduce illumination effects.

```
Original Image ■ ■■[σ]■■[kσ]■■[k²σ]■■... (Gaussian Scale Space) ■ ▼ DoG Pyramid (subtract adjacent scales) ■ ▼ Find Extrema (compare to 26 neighbors in 3D) ■ ▼ Reject low-contrast & edge pts (Hessian filter) ■ ▼ Assign Orientation → 128-D Descriptor
```

Numerical: How Hessian Filters Edge Keypoints

H=[[Dxx,Dxy],[Dxy,Dyy]] Tr(H)=Dxx+Dyy Det(H)=Dxx·Dyy−Dxy² Ratio test: Tr(H)²/Det(H) < (r+1)²/r where r=10 HIGH ratio → one eigenvalue >> other → EDGE → REJECT

■ Pros	■ Cons
• Scale & rotation invariant	• Slow — not real-time
• Robust to illumination changes	• 128-D vector = large memory
• 128-D descriptor is very distinctive	• Poor with motion blur

[Faster SIFT] SURF — Speeded-Up Robust Features

DEFINITION

Faster approximation of SIFT using integral images and box filters (Haar wavelets) instead of Gaussian convolutions.

- **Detection:** Fast Hessian using Box filters on integral images (approx. DoG)
- **Descriptor:** 64-D vector (4×4 grid, 4 Haar wavelet responses each) → 2× faster than SIFT
- **Orientation:** Dominant Haar wavelet response in 60° sliding window
- **U-SURF:** Upright SURF — skip orientation step if rotation invariance not needed (even faster)

	SIFT	SURF
Descriptor Size	128-D	64-D
Speed	Slow	3–7× faster
Scale invariant	■	■
Rotation invariant	■	■ (U-SURF: ■)
Patent	Expired	Proprietary

■ Pros	■ Cons
• Much faster than SIFT	• Less accurate than SIFT
• Good for real-time applications	• Poor with large viewpoint change
• Scale + rotation invariant	

[Object Detection] HOG — Histogram of Oriented Gradients

DEFINITION

Describes local shape/appearance by the distribution of intensity gradient orientations. Classic pedestrian/object detector.

Algorithm Steps

1. Divide image into cells (e.g. 8×8 pixels)
2. Compute gradient magnitude & direction for each pixel
3. Build 9-bin orientation histogram (0°–180°) per cell
4. Group cells into blocks (2×2 cells), normalize block histogram (L2 norm)
5. Concatenate all block histograms → final HOG feature vector
6. Feed to SVM classifier for detection

```
Image Window (64×128 for pedestrian) ■ ▼ Compute Gx, Gy → |G|, θ for each pixel ■ ▼ Each 8×8 cell → 9-bin orientation histogram ■ ▼ 2×2 cells = 1 block → normalize (contrast invariance) ■ ▼ Concatenate all block vectors → HOG descriptor ■ ▼ SVM → Detect / Not Detect
```

■ Pros	■ Cons
• Excellent for pedestrian detection	• NOT scale or rotation invariant
• Illumination invariant (block norm)	• Fixed image window size
• Simple + effective	• Fails with occlusion

Feature	SIFT	SURF	HOG
Scale Inv.	■	■	■
Rotation Inv.	■	■	■
Speed	Slow	Medium	Fast
Primary Use	Matching	Matching	Detection

[UNIT 02] Image Segmentation — Region, Edge, Graph, Texture

[■ EXAM HOT] Region Growing Segmentation

DEFINITION

Start from seed pixels and iteratively add neighboring pixels that satisfy a similarity criterion (intensity, color, texture). Grows regions outward from seeds.

Algorithm

1. **Seed Selection:** Choose initial seed pixel(s) inside the target region
2. **Initialize:** Add seeds to region R, add to queue
3. **Grow:** For each pixel in queue, examine neighbors (4-connected or 8-connected)
4. **Similarity Test:** If |I(neighbor)–I(seed)| < threshold T → add to region R
5. **Repeat:** Until no more pixels can be added (queue empty)

```
Seed (S) ■ [S]■■check N,E,S,W neighbors ■ ■■ |I(nbr) - I(seed)| ≤ T? ■■YES■■■ Add to Region R ■ Add to Queue ■■ NO → Skip Repeat until Queue empty
```

- **Seed Selection:** Critical — wrong seeds → wrong regions
- **Threshold T:** Low T → under-segmentation; High T → over-segmentation/leakage
- **Connectivity:** 4-connected (up/down/left/right) or 8-connected (+ diagonals)

■ Pros	■ Cons
• Simple and intuitive	• Leakage across weak boundaries
• User can guide via seeds	• Sensitive to noise and threshold
• Good for known defect regions	• Requires good seed selection

■ Exam Tip

Thermal defect/agricultural leaf/satellite flood: seed selection → grow with threshold → 8-connectivity → Graph-Cut to refine. Hybrid = standard exam answer!

[Edge-Based] Edge-Based Segmentation — Active Contours / Level Sets

DEFINITION

Uses detected edges/boundaries to delineate regions. Active contours (snakes) deform toward strong edges under energy minimization.

Active Contours (Snakes)

$$E_{total} = E_{internal} + E_{external} = \int(\alpha|v'|^2 + \beta|v''|^2)ds - \gamma|\nabla I|^2$$

- **E_internal:** Controls smoothness (α=elasticity, β=stiffness)
- **E_external:** Pulls contour toward image edges (gradient magnitude)
- Contour iteratively deforms to minimize total energy

1. Initialize contour around target (user draws rough outline)
2. Compute gradient-based edge force at each contour point
3. Update contour positions to reduce E_total
4. Iterate until convergence (contour stops moving)

Level Set Method (advanced)

- Represents contour as implicit zero-level of 3D function $\phi(x,y)$
- Handles topology changes (merging, splitting) automatically
- Better for complex shapes (CT/MRI vessel segmentation)

■ Pros	■ Cons
• Handles smooth boundaries	• Leaks through weak/faint edges
• Level sets handle topology changes	• Computationally intensive
• Works on CT/MRI/ultrasound	• Sensitive to initialization

[■ KEY TOPIC] Graph-Cut Segmentation

DEFINITION

Models image as a graph. Each pixel = node. Edges carry weights based on similarity. Min-cut algorithm partitions graph into foreground (Source) and background (Sink).

Graph Structure

- **Nodes:** Each pixel + Source (S) + Sink (T)
- **n-links (neighborhood):** Edges between adjacent pixels; weight = similarity (high = similar = don't cut)
- **t-links (terminal):** Edges to S (foreground probability) and T (background probability)

$$E(f) = \Sigma Dp(fp) + \lambda \cdot \Sigma Vpq(fp,fq) \quad [\text{Data term + Smoothness term}]$$

1. Build graph: pixels as nodes, similarity as edge weights
2. User marks foreground seeds (→ Source) and background seeds (→ Sink)
3. Assign t-link weights using color/intensity histograms (GMM)
4. Run max-flow / min-cut algorithm (Ford-Fulkerson)
5. Min-cut partition = segmentation boundary

```
S (Source/Foreground) |t-link |t-link [p1]■n■[p2]■n■[p3] ← pixels (nodes) |t-link |t-link T (Sink/Background) Min-cut: minimum weight edges to cut → FG from BG
```

■ Pros	■ Cons
• Global optimal solution	• Slow for large 3D volumes
• No leakage across weak edges	• Graph construction memory intensive
• User seeds refine result	• Requires seed points
• Handles intensity inhomogeneities	

■ Hybrid Pipeline (Exam Q5,Q6)

Region Growing (coarse) → Active Contour (boundary refinement) → Graph-Cut (global optimization). RG gives initial mask, AC aligns to edges, GC minimizes leakage globally.

[Clustering] Mean-Shift Segmentation

DEFINITION

Non-parametric density estimator. Pixels iteratively shift toward regions of higher density in feature space. Pixels converging to same mode = same segment.

m(x) = [ΣK((x–xi)/h)·xi] / [ΣK((x–xi)/h)] x ← m(x)

1. Represent each pixel as feature vector (L,a,b color + x,y position = 5D)
2. For each pixel, compute mean of nearby pixels within bandwidth h
3. Shift pixel toward computed mean
4. Repeat until shift < ε (pixel converged to density peak = mode)
5. Pixels sharing same mode → same segment

■ Pros	■ Cons
• No need to specify # clusters	• Bandwidth h sensitive — hard to tune
• Finds arbitrarily shaped regions	• Very slow O(n²) per iteration
• Robust to outliers	• Over-segments complex textures

[Probabilistic] Markov Random Fields (MRF)

DEFINITION

Probabilistic graphical model where each pixel's label depends only on its neighbors (Markov property). Used for restoration, texture classification, segmentation.

Energy Formulation

E(f) = Σψ_i(fi) + Σψ_ij(fi,fj) [Unary + Pairwise potentials]

- **Unary potential ψ_i:** Cost of assigning label f to pixel i (from observed intensity)
- **Pairwise potential ψ_ij:** Cost of adjacent pixels having different labels (encourages smoothness)
- **MAP-MRF:** Find labeling f that minimizes E(f)
- **Solved by:** Belief Propagation, Graph-Cut (for submodular energies)

■ Pros	■ Cons
• Principled probabilistic framework	• Complex to design & optimize
• Incorporates spatial context	• Slow inference
• Models complex label interactions	• Requires parameter tuning

[Texture] Texture Segmentation

DEFINITION

Segments regions based on texture properties (regular patterns, roughness, directionality) rather than just intensity.

- **Gabor Filters:** Sinusoid × Gaussian. Detects specific frequencies and orientations. Bank of filters → texture feature vector per pixel.
- **LBP (Local Binary Patterns):** Encode local texture by thresholding neighbors relative to center pixel → histogram.
- **Co-occurrence Matrix (GLCM):** Statistical measure of how often pairs of pixel values co-occur.
- **DWT (Discrete Wavelet Transform):** Multi-resolution decomposition → energy at each sub-band = texture feature.

```
Gabor Filter Bank: 0=0° 0=45° 0=90° 0=135° ■■■■ ■■■■ ■■■■ ■■■■ Each filter detects edges at specific orientation+frequency → Response vectors → Cluster → Segment by texture class
```

[UNIT 03] Pattern Analysis — Clustering, Classification, Dimensionality Reduction

[■ EXAM HOT] K-Means Clustering

DEFINITION

Partitions n data points into k clusters by iteratively assigning points to nearest centroid and recomputing centroids.

Objective: minimize J = Σ■ Σ■∈C■ ||x – μ■||²

1. Initialize: randomly choose k centroids μ■,...,μ■
2. Assignment: assign each point x to cluster with nearest centroid: c(x) = argmin_i ||x–μ■||²
3. Update: recompute centroid as mean: μ■ = (1/|C■|)·Σx∈C■ x
4. Repeat steps 2–3 until assignments don't change (convergence)

Numerical Example (Mock Q3)

```
Data: [15,20,10,25, 120,115,125,110, 130,140,135,145, 240,235,245,250] k=3, initial centroids: μ■=15(dark), μ■=125(mid), μ■=240(bright) Iter 1 Assignment: 15,20,10,25 → μ■ (dark) 120,115,125,110,130,140,135,145 → μ■ (mid) 240,235,245,250 → μ■ (bright) Update: μ■=(15+20+10+25)/4=17.5 μ■=127.5 μ■=242.5 → Converged! Result: 3 segments: dark / mid-tone / bright
```

■ Pros	■ Cons
• Simple and fast O(nkt)	• Must specify k in advance
• Scales to large data	• Sensitive to initialization
	• Assumes spherical clusters
	• Affected by outliers

[■ Q3 MOCK SOLUTION] Multi-Level Thresholding — Worked Numerical

```
Input matrix I: Rule: 15 120 130 240 g(x,y) = 0 if I ≤ 50 (Background) 20 115 140 235 128 if 50 < I ≤ 180 (Mid-tone) 10 125 135 245 255 if I > 180 (Foreground) 25 110 145 250 Result matrix g(x,y): Pixel Counts: 0 128 128 255 Background (0): 4 pixels (col 1) 0 128 128 255 Mid-tone (128): 8 pixels (cols 2+3) 0 128 128 255 Foreground (255): 4 pixels (col 4) 0 128 128 255 Total: 16 pixels ✓
```

[Robust K-Means] K-Medoids Clustering

DEFINITION

Like K-Means but cluster centers (medoids) must be actual data points. More robust to outliers.

1. Initialize: randomly select k data points as medoids
2. Assign each point to nearest medoid
3. For each cluster, try all points as potential new medoid — pick one minimizing total distance
4. Repeat until medoids don't change

	K-Means	K-Medoids
Center	Mean (may not exist in data)	Actual data point
Outlier Sensitivity	High	Low ■
Distance Metric	Euclidean only	Any metric ■
Speed	Fast O(nkt)	Slower O(n²kt)

■ Exam Answer (Practice Q7)

Industrial sensor data with outliers: K-Medoids (or GMM) for clustering → KNN or ANN for labeling abnormal samples with limited labeled data.

[Probabilistic] Gaussian Mixture Models (GMM)

DEFINITION

Models data as a mixture of K Gaussians. Each component has own mean (μ), covariance (Σ), and mixing weight (π). More flexible than K-Means.

p(x) = Σ■ π■·N(x|μ■,Σ■) where Σπ■ = 1

EM Algorithm (Expectation-Maximization)

1. **E-step:** Compute responsibility r_ik = P(cluster k | point x■) for each point
2. **M-step:** Update μ, Σ, π using weighted averages
3. Repeat until log-likelihood converges

■ Pros	■ Cons
• Soft assignments (probabilistic) • Handles elliptical clusters • Returns cluster probabilities	• Must specify k • Can converge to local optima • Slower than K-Means

[■ KEY TOPIC] Classifiers — Bayes, KNN, ANN + Learning Types

Learning Types

Type	Labeled Data	Use When	Example
Supervised	All labeled	Enough labels available	Disease classification
Unsupervised	None labeled	No labels, find patterns	Customer segmentation
Semi-supervised	Some labeled	Few labels + many unlabeled	Medical records (Q4 Mock)

Naive Bayes Classifier

P(C|x) = P(x|C)·P(C) / P(x) [Posterior = Likelihood × Prior / Evidence]

- Assumes features are independent (naive assumption)
- Fast, works well with small datasets
- Good when features actually independent (e.g. text classification)

K-Nearest Neighbors (KNN)

1. Compute distance from query point to all training points
2. Find K nearest neighbors
3. Assign majority class label among the K neighbors

dist(x,xi) = √Σ(x■–xi■)² [Euclidean distance]

- No training phase (lazy learner)
- K=1→noisy; K=large→smooth but misclassifies boundary
- Curse of dimensionality: performance drops in high dimensions

Artificial Neural Network (ANN)

1. Forward pass: input → hidden layers → output (softmax/sigmoid)
2. Compute loss (cross-entropy)
3. Backpropagation: compute gradients using chain rule
4. Update weights: w ← w – η·∂L/∂w

```
Input Layer Hidden Layer Output x■ ■■→ [N] ■■→ [N][N] ■■→ y■
x■ ■■→ [N] ■■→ [N][N] ■■→ y■ x■ ■■→ [N] ■■→ [N][N] ■■→ y■
Each N: output = σ(w·x + b)
```

	Bayes	KNN	ANN
Training	Fast	None (lazy)	Slow
Interpretable	■	■	■
Large Data	OK	Poor (slow)	■ Best
Non-linear	■	■ (local)	■ Best
Semi-supervised	OK	OK	■ Best

■ Mock Q4 Answer

Few labeled medical records → SEMI-SUPERVISED. Best: ANN (handles non-linear disease patterns, semi-supervised via self-training). Second: KNN (simple, interpretable).

Discriminant Function

g(x) = w■x + w■ [Linear discriminant]

- Divides feature space into decision regions
- g(x)>0 → Class 1; g(x)<0 → Class 2
- Linear discriminant: straight line boundary
- Quadratic discriminant: curved boundary (when covariances differ)
- Outliers pull decision boundary → use robust estimators or K-Medoids

[■ EXAM HOT] Dimensionality Reduction — PCA, LDA, ICA

PCA — Principal Component Analysis

DEFINITION

Unsupervised linear method. Projects data onto directions of maximum variance (principal components). No class labels needed.

1. Center data: x■ = x – μ (subtract mean)
2. Compute covariance matrix: C = (1/n)·X■■X■
3. Compute eigenvalues λ■ and eigenvectors v■ of C
4. Sort by eigenvalue (descending), select top k eigenvectors
5. Project: y = W■x■ where W = [v■|v■|...|vk]

LDA — Linear Discriminant Analysis

DEFINITION

Supervised linear method. Maximizes class separability: maximize ratio of between-class to within-class scatter.

J(W) = (W■SbW) / (W■SwW) [Between-class / Within-class scatter]

- Requires class labels → supervised
- Max (c–1) discriminant dimensions for c classes
- Better for classification than PCA

ICA — Independent Component Analysis

DEFINITION

Finds statistically INDEPENDENT components (not just uncorrelated like PCA). Used for blind source separation (EEG signals, mixed audio).

x = A·s ICA finds: W such that y = Wx has independent components

	PCA	LDA	ICA
Supervised	■	■	■
Goal	Max variance	Max class sep.	Independence
Components	Uncorrelated	Discriminant	Independent
Use	Compression, viz	Classification	Signal separation

■ Exam Answer (Practice Q8/Q9)

Surveillance system with high-dim features: Use PCA (unsupervised, fast, no labels) → KNN on reduced features. Cuts KNN cost from O(d·n) to O(k·n) where k<<d.

[UNIT 04] Motion Analysis — Background Subtraction, Optical Flow, KLT

[■ KEY TOPIC] Background Subtraction & Modeling

DEFINITION

Separates moving foreground objects from static background by comparing current frame to a background model.

Simple Frame Difference

D(x,y,t) = |I(x,y,t) – I(x,y,t–1)| Foreground if D > threshold

- Fast but misses slow objects, creates double contours

Background Modeling Methods

- **Running Average:** B(t) = α·I(t)+(1–α)·B(t–1). Simple, adapts slowly. α controls adaptation rate.
- **MOG (Mixture of Gaussians):** Model each pixel as GMM. Handles multiple background modes (waving trees, flicker). Best for outdoor surveillance.
- **MOG2:** Adaptive MOG — auto-selects number of components. OpenCV default.

```
t=1: bg model B■ t=2: new frame I■ ■ ▼ |I■(x,y) - B■(x,y)| > T?
■■■ YES → Foreground (moving object) ■■■ NO → Background →
Update B B(t) ← α·I(t) + (1-α)·B(t-1)
```

■ Pros	■ Cons
• Fast and simple • MOG handles dynamic backgrounds	• Fails with sudden illumination change • Shadow effects • Needs fixed camera

[■ EXAM HOT] Optical Flow

DEFINITION

Apparent motion of image pixels between consecutive frames. Gives a 2D velocity field (u,v) per pixel or feature point.

Optical Flow Constraint Equation

∂I/∂x·u + ∂I/∂y·v + ∂I/∂t = 0 → Ix·u + Iy·v = –It

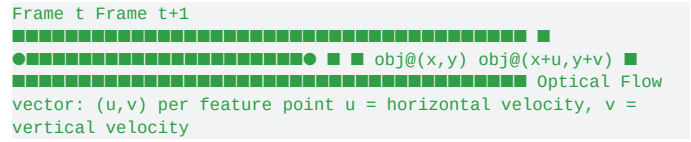
- Ix, Iy = spatial gradients; It = temporal gradient
- 2 unknowns (u,v), 1 equation → underdetermined (aperture problem)

Lucas-Kanade (Sparse / Feature-based)

- Assumes motion is constant within a small local window
- Solves: [u;v] = (A■A)■¹A■b where A=[Ix,Iy] for each pixel in window
- Works well for sparse feature points (corners)

Horn-Schunck (Dense)

- Adds global smoothness constraint: minimize ||∇u||²+||∇v||²
- Computes flow at EVERY pixel (dense field)



■ Pros	■ Cons
• Estimates per-pixel motion	• Fails with large motion
• Works for moving camera	• Aperture problem
• No explicit segmentation needed	• Dense OF is slow

■ Exam Tip

Background Subtraction = WHAT moved (binary foreground mask). Optical Flow = HOW it moved (velocity vectors). Use both: BS detects movers, OF estimates direction and speed.

[Feature Tracker] KLT Tracker (Kanade-Lucas-Tomasi)

DEFINITION

Feature tracker that selects good features (high gradient regions = corners) and tracks them using Lucas-Kanade optical flow across frames.

Algorithm

1. Select feature points using Shi-Tomasi criterion: λ_min of structure tensor > threshold
2. For each feature, compute Lucas-Kanade displacement (u,v) to next frame
3. Update feature position: x' = x+u, y' = y+v
4. Check tracking quality: if tracking error > threshold → delete feature
5. Add new features if too few remain

Shi-Tomasi Feature Quality

Quality(p) = min(λ■, λ■) > threshold [Both eigenvalues large = good corner]

■ Pros	■ Cons
• Fast and reliable tracker	• Fails with large motion
• Works in real-time	• Needs good texture (can't track flat regions)
• Handles affine deformations	

[Advanced] Spatio-Temporal Analysis & Dynamic Stereo

DEFINITION

Extends spatial analysis to time dimension. Video = 3D volume (x,y,t). 3D features capture both spatial appearance and temporal motion patterns.

Spatio-Temporal Interest Points (STIP)

- Extend Harris detector to 3D: detect corner-like structures in (x,y,t)
- Used for action recognition in videos

Dynamic Stereo

- Stereo matching applied to video sequences over time
- Use both spatial stereo disparity AND temporal flow constraints
- Better depth estimation for moving objects

Motion Parameter Estimation

- **Translational motion:** Uniform (u,v) across frame — global motion
- **Affine motion:** 6-parameter model: u=a■+a■x+a■y, v=a■+a■x+a■y
- **Homography:** 8-parameter projective model for planar scenes

■ Exam Answer Frameworks — QUICK REFERENCE

[MUST READ] Standard Answer Templates for Common Question Types

Template: Feature Extraction Pipeline (warehouse/robot questions)

1. Edge Detector (Canny) → detect all edges → input to Hough
2. Hough Transform → detect straight lines (shelves, walls)
3. Corner Detector (Harris) → find shelf corners, anchor points
4. Descriptor (SIFT/HOG) → identify/classify objects on shelves

Template: Segmentation Hybrid Pipeline

1. Noise removal: Gaussian blur or median filter
2. User provides seeds → Region Growing (coarse segmentation)
3. Active Contour / Level Set → refine boundary to edges
4. Graph-Cut → global optimization, prevents leakage
5. Output: clean binary mask ready for downstream detection

Template: Pattern Analysis Pipeline

1. Extract features from data (intensity, texture, etc.)
2. Dimensionality Reduction: PCA (unsupervised) or LDA (supervised)
3. Clustering: K-Means (fast) or GMM (soft) or K-Medoids (robust)
4. Classification: KNN (simple) or ANN (complex) → discriminant function for decision

Template: Surveillance / Motion Pipeline

1. Background Subtraction (MOG2) → detect WHAT moved (foreground mask)
2. Optical Flow (Lucas-Kanade) → detect HOW they moved (velocity vectors)
3. KLT Tracker → track individual people/objects across frames
4. Dimensionality reduction (PCA) on features → KNN for person identification

Key Formulas to Memorize

```
Gradient: |G| = √(Gx²+Gy²) θ = arctan(Gy/Gx) Canny Hysteresis: • Edge if pixel > T_high • Non-edge if pixel < T_low • Maybe edge if T_low < pixel < T_high (keep only if connected to strong edge) Harris: R = det(M) − k·trace(M)² k ≈ 0.04–0.06 Corner: R>>0 | Edge: R<<0 | Flat: R≈0 SIFT: 4×4 cells × 8 bins = 128-D vector Hough Line: ρ = x·cos(θ) + y·sin(θ) Optical Flow: Ix·u + Iy·v + It = 0 K-Means: J = Σ■ Σ■∈C■ ||x − μ■||² Bayes: P(C|x) = P(x|C)·P(C) / P(x)
```